

CoderPad Questions and Answers - Review Notes

Prepared from the questions shared in this chat. Use this as a study and explanation guide so you can justify each answer in your own words.

Question 2 / 20 - Python 3

Question: Which Python instruction extracts all template variables between double curly brackets from a multiline string?

Answer:

```
re.findall('{{[a-zA-Z0-9_]*}}', text)
```

Why: findall returns every captured match as a list. The capture group extracts only the variable name between {{ and }}.

Question 3 / 20 - Python 3

Question: Which instruction can be used to add the number 5 to a set named values?

Answer:

```
values.add(5)
```

Why: Sets use add() to insert a single element. append() is for lists.

Question 4 / 20 - Python 3

Question: Among two class design solutions involving CanFly, Plane, and Bird, which one is best designed?

Answer:

Solution #1

Why: Plane and Bird are different object types that share a behavior. A CanFly interface/base abstraction is better than making Bird inherit from Plane.

Question 5 / 20 - Python 3

Question: What does the command pip install . do in a Python project?

Answer:

Installs the current project as a package, using setup.py or pyproject.toml.

Why: The dot means the current directory. pip reads the project metadata and installs that local project.

Question 6 / 20 - Python 3

Question: Given an object with instance attributes and class attribute changes, which statements about obj.a and obj.b are true?

Answer:

```
obj.a evaluates to 'hello'  
obj.b evaluates to None
```

Why: The instance attribute obj.a = 'hello' shadows the class attribute MyClass.a = 'john'. Then obj.b = None directly changes that instance attribute.

Question 7 / 20 - Python 3

Question: How do you get the types associated with function parameters and return value from a function definition?

Answer:

```
buy_product.__annotations__
```

Why: Function annotations are stored in the `__annotations__` dictionary, including the 'return' key.

Question 8 / 20 - Python 3

Question: You can use the time module to do what?

Answer:

```
Call the sleep function to pause your program.
```

Why: `time.sleep(seconds)` pauses execution for the given number of seconds.

Question 9 / 20 - Python 3

Question: Given two async coroutines using `asyncio.sleep()` and `asyncio.gather()`, what will the program print and after how long?

Answer:

```
The program will print ['Result A', 'Result B'] after 20 seconds.
```

Why: `asyncio.gather` runs coroutines concurrently. The total time is the longest await, not the sum: `max(10, 20) = 20` seconds.

Question 10 / 20 - Python 3

Question: Which of the listed expressions evaluate to True in a conditional expression?

Answer:

```
not {}  
[None]  
[1, 2, 3]
```

Why: Empty containers, None, and 0 are falsy. `not {}` is true. Non-empty lists are truthy even if the element is None.

Question 11 / 20 - Python 3

Question: Which instructions let you sort `arr = [0,3,1,2,4]` in ascending order?

Answer:

```
arr.sort()  
arr = sorted(arr)
```

Why: `arr.sort()` sorts in place. `sorted(arr)` returns a new sorted list, so assigning it back is valid.

Question 12 / 20 - Python 3

Question: Which parameter, after `pip install`, will clone a Git repository and install it as a package?

Answer:

```
git+https://github.com/numpy/numpy@v2.1.3
```

Why: The `git+https` form tells pip to install directly from a Git repository at the requested tag or revision.

Question 13 / 20 - AI Agent Engineering on Microsoft Foundry and Azure

Question: Deploying an AI agent service to Azure Container Apps: it calls Azure OpenAI, must avoid secrets in code/images, and must support key rotation without redeploying code. Which approach is best?

Answer:

```
Use a managed identity for the Container App to access Azure Key Vault; grant get permission to the specific secrets; inject secrets at runtime via Key Vault references or startup fetch; never store the key in
```

the container image or app settings.

Why: Managed identity removes hardcoded credentials. Key Vault centralizes secret storage and rotation.

Question 14 / 20 - AI Agent Engineering on Microsoft Foundry and Azure

Question: Designing an enterprise agent that processes sensitive customer data in Azure Container Apps and calls Azure OpenAI. Security requires minimizing public internet exposure. Which design is best?

Answer:

Use a private endpoint/VNet integration where possible, and call Azure OpenAI from the Container App using managed identity or a Key Vault key without exposing the OpenAI endpoint publicly.

Why: Private networking minimizes public exposure. Managed identity/Key Vault avoids embedding credentials.

Question 15 / 20 - AI Agent Engineering on Microsoft Foundry and Azure

Question: An agent sometimes returns the correct answer but takes 3-4x longer. You suspect retries due to schema validation failures and retrieval timeouts. You need diagnosis without leaking PII. Which observability approach is best?

Answer:

Emit distributed traces that include prompt version, model deployment name, tool call arguments/results with sensitive fields redacted, retrieval IDs/scores, and retry/loop counters; correlate with request IDs for incident triage.

Why: This gives enough operational detail to debug latency and retry loops while redacting sensitive data.

Question 16 / 20 - AI Agent Engineering on Microsoft Foundry and Azure

Question: An agent uses function calling to create IT tickets. Some calls fail because the model emits invalid JSON or invalid enum values. You want automatic recovery without creating incorrect tickets. Which change improves reliability?

Answer:

Configure strict structured outputs / JSON schema for the tool call arguments; validate and on validation failure return a structured error that the model can correct, with a low retry limit.

Why: Schema validation prevents bad tickets. A structured error lets the model retry safely, while a low retry limit controls loops.

Question 17 / 20 - AI Agent Engineering on Microsoft Foundry and Azure

Question: A customer-facing agent has P95 SLO < 2.5s and strict budget. It uses RAG and occasional tools. Complex cases benefit from larger model, but most queries are simple. Which design best balances latency, accuracy, and cost?

Answer:

Introduce a two-stage policy: use a small/faster model for routing and simple FAQ answers; escalate to a larger model only when retrieval confidence is low or complex tool use is required; cap tool retries to control tail latency.

Why: This keeps common-path latency and cost low while preserving accuracy for difficult cases.

Quick Answer Key

#	Correct Answer / Design Pattern
1	<code>re.findall('{{{[a-zA-Z0-9_]*}}}', text)</code>
2	<code>values.add(5)</code>
3	Solution #1
4	Installs the current project as a package, using <code>setup.py</code> or <code>pyproject.toml</code> .
5	<code>obj.a</code> evaluates to 'hello' <code>obj.b</code> evaluates to None
6	<code>buy_product.__annotations__</code>
7	Call the <code>sleep</code> function to pause your program.
8	The program will print ['Result A', 'Result B'] after 20 seconds.
9	not {} [None] [1, 2, 3]
10	<code>arr.sort()</code> <code>arr = sorted(arr)</code>
11	<code>git+https://github.com/numpy/numpy@v2.1.3</code>
12	Use a managed identity for the Container App to access Azure Key Vault; grant get permission to the specific secrets; inject secrets at runtime via Key Vault references or startup fetch; never store the key in the container image or app settings.
13	Use a private endpoint/VNet integration where possible, and call Azure OpenAI from the Container App using managed identity or a Key Vault key without exposing the OpenAI endpoint publicly.
14	Emit distributed traces that include prompt version, model deployment name, tool call arguments/results with sensitive fields redacted, retrieval IDs/scores, and retry/loop counters; correlate with request IDs for incident triage.
15	Configure strict structured outputs / JSON schema for the tool call arguments; validate and on validation failure return a structured error that the model can correct, with a low retry limit.
16	Introduce a two-stage policy: use a small/faster model for routing and simple FAQ answers; escalate to a larger model only when retrieval confidence is low or complex tool use is required; cap tool retries to control tail latency.